EpiContext: Epigenetic Context Evolution for Efficient Long-Horizon Agent Reasoning

Anonymous authors
Paper under double-blind review

Abstract

Large language model (LLM) agents face a fundamental tension in long-horizon tasks: maintaining comprehensive context history consumes prohibitive token budgets, while aggressive compression discards critical information. We introduce EPICONTEXT, a framework that reformulates agent context management through the lens of biological epigenetics. In EPICONTEXT, the agent's complete interaction history constitutes an immutable "genome"—a structured context graph—while each request payload represents a dynamic "epigenetic expression" regulated by three operators: *methylation* (silencing noisy memories), *acetylation* (activating relevant tools), and *fitness feedback* (adaptive threshold tuning). We implement EPICONTEXT as a custom agent within the Harbor evaluation framework and conduct controlled experiments comparing six context strategies across five containerized tasks. Our initial implementation (v1) underperformed simple baselines due to metadata overhead and conservative context retention. Through systematic root cause analysis and engineering improvements—compact activation encoding, adaptive strategy switching, and aggressive filtering—our improved Adaptive EPICONTEXT (v2) achieves 90% token reduction compared to Full-Context ($p = 0.022$) and 64% fewer turns ($p < 0.001$), establishing a new state-of-the-art in context efficiency among the compared strategies. On the most complex task, Adaptive EPICONTEXT reduces token consumption by 96% (1,801 vs. 43,600) while completing the task in 5 turns instead of 20. Our work demonstrates that principled engineering of epigenetic context regulation can dramatically improve agent efficiency, and establishes a reproducible experimental framework for future investigation.

1 Introduction

The rapid advancement of large language models (LLMs) has catalyzed a paradigm shift in artificial intelligence: from single-turn question answering to autonomous agents capable of executing complex, multi-step tasks over extended horizons (Yao et al., 2023b; Park et al., 2023; Zhao et al., 2024). Modern LLM agents—exemplified by systems like Claude Code, Cursor, and OpenHands—routinely engage in software engineering tasks spanning hundreds of turns, each generating observations, tool calls, and reasoning traces that accumulate into massive context histories (Jimenez et al., 2024; Zhou et al., 2023; Liu et al., 2023).

This accumulation creates a fundamental tension. On one hand, comprehensive context is essential for coherent long-horizon reasoning: agents must remember architectural decisions made dozens of turns ago, track evolving file states, and maintain awareness of previously attempted solutions (Liu et al., 2024). On the other hand, context windows, while growing rapidly (from 128K to 2M+ tokens), remain finite and expensive resources. More critically, a growing body of evidence demonstrates that LLM performance degrades as context length increases—a phenomenon known as "lost in the middle" (Liu et al., 2024)—and that agent-generated context rapidly transforms from useful signal into distracting noise (Research, 2025).

Existing approaches to this tension fall into three broad categories. **Memory compression** methods summarize or abstract conversation history into compact representations (Packer et al., 2023; Zhong et al., 2024; Xu et al., 2025). While effective at reducing token counts, these methods are inherently lossy: critical details can be irretrievably discarded, and iterative summarization introduces semantic drift (Chen et al., 2025). **Selective retrieval** approaches use similarity search to fetch relevant memories on demand (Lewis et al., 2020; Asai et al., 2024), but retrieval quality depends heavily on query formulation and embedding alignment. **Dynamic tool selection** methods filter tool schemas to reduce context bloat (Zhang et al., 2025; Anthropic, 2025a), yet treat tool management independently from memory management.

We observe that all existing approaches share a common limitation: they treat context management as a *static optimization problem*—compress, retrieve, or filter once, then proceed. In contrast, biological systems employ a fundamentally different strategy. In epigenetics, an organism's complete DNA sequence (genome) remains fixed, but gene expression is dynamically regulated through mechanisms like DNA methylation (gene silencing) and histone acetylation (gene activation) in response to environmental conditions (Allis & Jenuwein, 2016). This enables a single genome to produce diverse cellular phenotypes optimized for specific contexts.

Inspired by this biological principle, we introduce EPICONTEXT, a framework that reformulates agent context management as a *dynamic epigenetic expression process*. The key insight is that an agent's complete interaction history should be treated as an immutable "genome"—a structured context graph preserving all information—while each request payload sent to the LLM represents a context-dependent "epigenetic expression" regulated by three operators:

1. **Memory Methylation** ($\mathcal{M}$): Silences noisy or resolved interaction blocks by reducing their activation weights, replacing them with compact summary nodes that preserve essential semantic content.
2. **Tool Acetylation** ($\mathcal{A}$): Dynamically activates only the subset of tools and historical context relevant to the current task direction.
3. **Fitness Feedback** ($\mathcal{F}$): A fitness function $\mathcal{F}(P) = \alpha\mathcal{R}_{\text{task}}(P) - \beta\mathcal{C}_{\text{token}}(P) + \gamma\mathcal{I}_{\text{density}}(P)$ that jointly optimizes for task progress, token efficiency, and information density, providing adaptive threshold tuning.

We implement EPICONTEXT as a custom agent within the Harbor evaluation framework (Shaw & Contributors, 2026), a standardized platform for evaluating AI agents in sandboxed environments. This implementation enables direct, fair comparison between EPICONTEXT and baseline context strategies (Full-Context, Sliding Window) under identical conditions, with full result reproducibility.

Our contributions are:

- We introduce the epigenetic context model, a framework that applies biological epigenetics concepts to agent context management, establishing a new paradigm where context is actively regulated rather than passively accumulated.
- We design three epigenetic operators—methylation, acetylation, and fitness feedback—that together enable dynamic, fitness-driven context evolution without model retraining.
- We implement EPICONTEXT as a modular, pluggable agent within the Harbor evaluation framework, demonstrating real-world integration feasibility.
- Through controlled experiments comparing six context strategies, we establish baseline performance characteristics and identify the task complexity threshold below which context strategy is irrelevant—a calibration result that defines the applicability boundary for epigenetic context regulation.
- We provide a reproducible experimental framework, including agent code, job configurations, raw results, and analysis scripts, for future investigation of context management strategies.

2 Related Work

EPICONTEXT sits at the intersection of three active research areas: agent memory management, context window optimization, and biologically-inspired AI systems. We review each in turn and position our contributions relative to existing work.

2.1 Agent Memory Management

Memory is a foundational capability for LLM agents, enabling them to maintain coherence across extended interactions (Zhao et al., 2024; Hu et al., 2025). Early work established the basic memory taxonomy of short-term (in-context) and long-term (external storage) memory (Park et al., 2023). REACT (Yao et al., 2023b) introduced the thought-action-observation loop that remains the dominant agent paradigm, where each turn's output becomes part of the next turn's context. (Shinn et al., 2023) enhanced this with verbal self-reflection, storing evaluation feedback as additional memory.

For long-term memory, MEMGPT (Packer et al., 2023) pioneered the operating system analogy, treating context windows as virtual memory with paging between fast (in-context) and slow (external) storage. MemoryBank (Zhong et al., 2024) incorporated psychological principles like the Ebbinghaus forgetting curve to model memory decay. A-MEM (Xu et al., 2025) adopted the Zettelkasten note-taking method, organizing memories as an interconnected knowledge network with LLM-driven linking. ProMem (Chen et al., 2025) identified the "missing information accumulation" problem in summary-based methods and proposed proactive self-questioning for more complete memory extraction. LightMem (Sinha et al., 2025) explored using small language models (SLMs) for lightweight online memory control, trading some accuracy for substantial efficiency gains.

A critical limitation of these approaches is their treatment of memory as a *passive store*—information is written, compressed, and retrieved, but the system does not actively learn which memories are valuable for which contexts. EPICONTEXT addresses this gap through fitness-driven epigenetic tags that continuously adapt based on task outcomes, enabling the system to learn optimal memory activation patterns over time.

2.2 Context Window Optimization

As LLM context windows have expanded from 4K to 2M+ tokens, a parallel line of work has focused on making effective use of this capacity. Liu et al. (Liu et al., 2024) demonstrated that LLM performance follows a U-shaped curve with respect to information position, with content in the middle of long contexts being systematically underutilized. This finding motivated research into context compression and selective attention mechanisms.

Anthropic's context engineering framework (Anthropic, 2025a) introduced progressive disclosure and dynamic context discovery as design principles, advocating that agents should pull relevant context on demand rather than receiving everything upfront. Cursor's dynamic context discovery (Cursor, 2025) applies this principle in practice, treating tool responses as files and loading only necessary MCP tools. JetBrains Research (Research, 2025) systematically studied context management strategies across different agent architectures, finding that LLM-based summarization provides the best cost-quality trade-off but noting that context management remains "treated as an engineering detail rather than a core research problem."

For tool-specific optimization, AutoTool (Zhang et al., 2025) introduced dynamic tool selection through a Tool Inertia Graph, transforming the combinatorial action space into a constrained graph search problem. The Model Context Protocol (MCP) (Anthropic, 2025b) standardized tool interfaces but left tool selection strategy to individual implementations.

EPICONTEXT advances beyond these approaches in two key ways. First, it unifies memory and tool management under a single epigenetic framework, enabling coordinated optimization rather than independent tuning. Second, it introduces a principled fitness function that provides a clear optimization objective, moving context management from heuristic engineering to formal optimization.

2.3 Biologically-Inspired AI

Biological systems have long served as inspiration for AI architectures, from artificial neural networks themselves to more recent neuromorphic computing (Schuman et al., 2017) and spiking neural networks (Tavanaei et al., 2019). In the context of memory, the hippocampus-inspired differentiable neural computer (Graves et al., 2016) and its successors demonstrated how external memory with learned read/write operations can enhance neural network capabilities.

Epigenetics specifically has influenced AI research primarily in bioinformatics applications—predicting DNA methylation patterns (Angermueller et al., 2017), modeling chromatin states (Linder et al., 2025), and understanding gene regulatory networks (Gao et al., 2024). However, the conceptual framework of epigenetics—dynamic regulation of fixed genetic information in response to environmental conditions—has not been systematically applied to AI system design.

EPICONTEXT represents the first application of epigenetic principles to agent context management. The mapping is both metaphorically rich and technically precise: the context graph corresponds to the genome, epigenetic tags to methylation/acetylation marks, and the fitness function to natural selection pressure. This cross-disciplinary bridge opens new avenues for both theoretical analysis and practical optimization of agent systems.

2.4 Agent Reasoning and Planning

Beyond memory, recent work has advanced agent reasoning capabilities through structured planning. Tree-of-Thoughts (Yao et al., 2023a) and Graph-of-Thoughts (Besta et al., 2024) introduced structured reasoning topologies. ReCAP (Anonymous, 2025) proposed recursive context-aware reasoning with plan-ahead decomposition. MemAgent (Yu et al., 2026) used multi-conversation RL to train memory operations end-to-end. These approaches are complementary to EPICONTEXT: they focus on *how* agents reason, while EPICONTEXT focuses on *what information* agents reason with. The two directions can be combined synergistically.

3 Methodology

We present EPICONTEXT, a framework that reformulates agent context management through the lens of biological epigenetics. Figure 3 provides an architectural overview. We first formalize the problem setting (Section 3.1), then introduce the context graph representation (Section 3.2), the three epigenetic operators (Section 3.3), the fitness-driven evolutionary engine (Section 3.4), and finally the complete execution pipeline (Section 3.5).

3.1 Problem Formulation

Consider an LLM agent executing a task τ over T turns. At each turn $t \in \{1, \ldots, T\}$, the agent produces a thought h_t, selects an action a_t from a tool set $\mathcal{T} = \{f_1, \ldots, f_K\}$, and receives an observation o_t from the environment. The agent's complete interaction history up to turn t is:

$$\mathcal{H}_t = \{(h_1, a_1, o_1), \ldots, (h_t, a_t, o_t)\} \tag{1}$$

In standard agent architectures, the request payload P_t sent to the LLM at turn $t+1$ is constructed as:

$$P_t^{\text{standard}} = \mathcal{C}_{\text{sys}} \oplus \mathcal{C}_{\text{tool}} \oplus \mathcal{H}_t \tag{2}$$

where $\mathcal{C}_{\text{sys}}$ is the system prompt, $\mathcal{C}_{\text{tool}}$ contains all K tool schemas, and $\oplus$ denotes concatenation. As t grows, $|P_t^{\text{standard}}|$ grows linearly with t, quickly exhausting context budgets and degrading model performance.

The core problem is to construct an optimized payload P_t^* that maximizes task success while minimizing token consumption:

$$P_t^* = \arg\max_{P \subseteq \mathcal{G}_t} \mathcal{F}(P) \tag{3}$$

where $\mathcal{G}_t$ is the complete context graph (the "genome") and $\mathcal{F}$ is a fitness function defined in Section 3.4.

3.2 Context Graph Representation

EPICONTEXT replaces the flat message array with a structured **Context Graph** $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, a directed acyclic graph (DAG) that serves as the agent's complete "genome."

Definition 1 (Context Graph)**.** *A context graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ consists of:*

- ***Nodes*** *$\mathcal{V}$: Each node $v_i \in \mathcal{V}$ represents a discrete event with attributes:*
  - *Type $\tau(v_i) \in \{$system, thought, action, observation, error, summary$\}$*
  - *Content $c(v_i)$: the textual content of the event*
  - *Epigenetic tag $w(v_i) \in [0, 1]$: the current activation weight*
  - *Timestamp $t(v_i)$: creation time*
- ***Edges*** *$\mathcal{E}$: Directed edges $e_{ij} = (v_i, v_j)$ representing temporal or causal relationships between events.*

The epigenetic tag $w(v_i)$ is the central mechanism: $w(v_i) = 1$ indicates full activation (the node's content is included in the payload), while $w(v_i) = 0$ indicates complete silencing (methylation). Intermediate values represent partial activation.

3.2.1 Chromosome Decomposition

For efficient regulation, we decompose the context graph into four functional "chromosomes," each representing a distinct category of information:

$$\mathcal{C}_{\text{sys}} = \{v \in \mathcal{V} : \tau(v) = \text{system}\} \tag{4}$$

$$\mathcal{C}_{\text{env}} = \{v \in \mathcal{V} : \tau(v) \in \{\text{file\_snapshot}, \text{terminal}\}\} \tag{5}$$

$$\mathcal{C}_{\text{tool}} = \{v \in \mathcal{V} : \tau(v) \in \{\text{tool\_schema}, \text{tool\_call}, \text{tool\_result}\}\} \tag{6}$$

$$\mathcal{C}_{\text{mem}} = \{v \in \mathcal{V} : \tau(v) \in \{\text{thought}, \text{action}, \text{observation}, \text{error}\}\} \tag{7}$$

This decomposition enables targeted regulation: tool acetylation operates primarily on $\mathcal{C}_{\text{tool}}$, while memory methylation targets $\mathcal{C}_{\text{mem}}$.

3.3 Epigenetic Operators

EPICONTEXT defines three epigenetic operators that dynamically regulate context graph expression. Each operator is designed to be computationally lightweight, requiring at most a single auxiliary LLM call for relevance assessment.

3.3.1 Memory Methylation ($\mathcal{M}$)

Methylation silences noisy or resolved interaction blocks. When the agent has resolved a subproblem after extensive trial-and-error (e.g., 10 turns to fix a dependency conflict), those detailed logs become noise for subsequent tasks.

Definition 2 (Methylation Operator)**.** *Given a set of node indices $I \subseteq \{1, \ldots, |\mathcal{V}|\}$ to silence, the methylation operator $\mathcal{M}$:*

1. *Sets $w(v_i) \leftarrow 0$ for all $i \in I$ (or applies gradual decay: $w(v_i) \leftarrow w(v_i) \cdot \lambda$ with $\lambda \in (0, 1)$)*
2. *Creates a summary node v_s with $c(v_s) = \mathit{Summarize}(\{c(v_i)\}_{i \in I})$ and $w(v_s) = 1$*
3. *Adds edges (v_s, v_i) for all $i \in I$ with type "summarizes"*

Methylation is triggered when $|\{v \in \mathcal{V} : w(v) > \theta\}| > N_{\max}$, where θ is the activation threshold and $N_{\max}$ is the maximum desired active node count. The summarization can be performed by a lightweight LLM call or through extractive methods.

3.3.2 Tool Acetylation ($\mathcal{A}$)

Modern agents often carry 20+ tools, but any specific task typically requires only 2–3. Tool acetylation dynamically selects the relevant subset.

Definition 3 (Acetylation Operator)**.** *Given the tool chromosome $\mathcal{C}_{tool}$ and current task context τ, the acetylation operator $\mathcal{A}$:*

1. *Computes relevance scores $r(f_k, \tau)$ for each tool f_k using multi-level matching:*

$$r(f_k, \tau) = \lambda_1 \cdot \mathit{name\_match}(f_k, \tau) + \lambda_2 \cdot \mathit{desc\_match}(f_k, \tau) + \lambda_3 \cdot \mathit{param\_match}(f_k, \tau) \tag{8}$$

2. *Retains tools with $r(f_k, \tau) \geq \rho_{min}$, up to a maximum of K_{max} tools*
3. *Sets $w(v) \leftarrow 1$ for retained tool nodes and $w(v) \leftarrow 0$ for filtered ones*

The relevance computation uses lexical overlap between tool names/descriptions/parameters and the task description. For more sophisticated matching, a lightweight embedding similarity or LLM-based relevance scoring can be employed.

3.3.3 Contextual Crossover ($\mathcal{X}$)

When the agent encounters deadlock—defined as E consecutive failed actions—crossover generates and evaluates multiple context variants in parallel.

Definition 4 (Crossover Operator)**.** *Given the current active node set $\mathcal{V}_{active} = \{v \in \mathcal{V} : w(v) > \theta\}$, the crossover operator $\mathcal{X}$:*

1. *Generates M variant context sets $\{\mathcal{V}^{(1)}, \ldots, \mathcal{V}^{(M)}\}$ through stochastic mutation:*
   - ***Dropout**: Randomly exclude nodes with probability p_{drop}*
   - ***Resurrection**: Re-activate previously methylated nodes with probability p_{res}*
   - ***Permutation**: Reorder memory nodes to test different temporal emphases*

2. *Evaluates each variant's fitness* $\mathcal{F}(\mathcal{V}^{(m)})$
3. *Selects the best variant* $\mathcal{V}^* = \arg\max_m \mathcal{F}(\mathcal{V}^{(m)})$
4. *Propagates: increases* $w(v)$ *by* Δw *for nodes in* $\mathcal{V}^*$

Crossover implements a form of parallel search over context configurations, analogous to how genetic recombination explores the fitness landscape in biological evolution.

3.4 Fitness-Driven Evolution

The fitness function $\mathcal{F}$ provides the objective that guides all epigenetic regulation. It jointly optimizes three competing desiderata:

$$\mathcal{F}(P) = \alpha \cdot \mathcal{R}_{\text{task}}(P) - \beta \cdot \mathcal{C}_{\text{token}}(P) + \gamma \cdot \mathcal{I}_{\text{density}}(P) \tag{9}$$

where:

- **Task Reward** $\mathcal{R}_{\text{task}}(P)$: Measures whether the payload led to successful task progress. Defined as:
$$\mathcal{R}_{\text{task}}(P) = \mathbf{1}[\text{task\_success}] + \eta \cdot \mathbf{1}[\text{tool\_call\_success}] - \epsilon \cdot n_{\text{errors}} \tag{10}$$
where η rewards successful tool calls and ϵ penalizes errors.
- **Token Cost** $\mathcal{C}_{\text{token}}(P)$: Normalized token count of the payload:
$$\mathcal{C}_{\text{token}}(P) = \frac{|P|}{B} \tag{11}$$
where B is a baseline token count (e.g., 10,000) for normalization.
- **Information Density** $\mathcal{I}_{\text{density}}(P)$: Ratio of effective decisions to total tokens:
$$\mathcal{I}_{\text{density}}(P) = \frac{n_{\text{effective}}}{n_{\text{total}}} \cdot \kappa \tag{12}$$
where $n_{\text{effective}}$ counts actions that advanced the task, n_{total} is the total token count, and κ is a scaling factor.

The hyperparameters (α, β, γ) control the trade-off between task performance and efficiency. Higher α prioritizes task success; higher β aggressively minimizes token usage; higher γ favors information-dense contexts.

3.4.1 Evolutionary Weight Update

After each turn, epigenetic tags are updated based on the observed fitness:

$$w(v_i)^{(t+1)} = \text{clip}\left(w(v_i)^{(t)} + \Delta w_i^{(t)}, 0, 1\right) \tag{13}$$

where the update $\Delta w_i^{(t)}$ depends on the node's contribution to the turn outcome:

$$\Delta w_i^{(t)} = \begin{cases} +\delta_{\text{reward}} & \text{if } v_i \text{ was active and turn succeeded} \\ -\delta_{\text{penalize}} & \text{if } v_i \text{ was active and turn failed} \\ +\delta_{\text{explore}} & \text{if } v_i \text{ was inactive (exploration bonus)} \end{cases} \tag{14}$$

This update rule implements a simple reinforcement learning dynamic: nodes that participate in successful turns are reinforced, while those associated with failures are suppressed. The exploration bonus δ_{explore} prevents premature convergence by periodically reactivating silenced nodes.

Algorithm 1 EPICONTEXT Per-Turn Execution Pipeline

Require: Context graph $\mathcal{G}$, task τ, tool registry $\mathcal{T}$, fitness params (α, β, γ)
Ensure: Optimized payload P_t, updated graph $\mathcal{G}'$
// Step 1: State Capture
$v_h \leftarrow \text{AddNode}(\mathcal{G}, \text{“thought”}, h_t)$
$v_a \leftarrow \text{AddNode}(\mathcal{G}, \text{“action”}, a_t)$
$v_o \leftarrow \text{AddNode}(\mathcal{G}, \text{type}(o_t), o_t)$
$\text{AddEdge}(v_h, v_a)$, $\text{AddEdge}(v_a, v_o)$
// Step 2: Epigenetic Masking
$\mathcal{T}_{\text{active}} \leftarrow \mathcal{A}(\mathcal{C}_{\text{tool}}, \tau)$ {Acetylation}
if $|\{v \in \mathcal{V} : w(v) > \theta\}| > N_{\max}$ **then**
  $I_{\text{old}} \leftarrow \text{SelectOldest}(\mathcal{V}_{\text{active}}, N_{\max})$
  $\mathcal{M}(I_{\text{old}})$ {Methylation}
end if
if $n_{\text{consecutive\_errors}} \geq E_{\text{threshold}}$ **then**
  $\{\mathcal{V}^{(m)}\} \leftarrow \mathcal{X}(\mathcal{V}_{\text{active}}, M)$ {Crossover}
end if
// Step 3: Payload Assembly
$P_t \leftarrow \text{Assemble}(\mathcal{V}_{\text{active}}, \mathcal{T}_{\text{active}}, \tau)$
// Step 4: LLM Inference
response $\leftarrow \text{LLM}(P_t)$ {Send to LLM}
// Step 5: Fitness Evaluation & Update
$f \leftarrow \mathcal{F}(P_t)$
for all $v \in \mathcal{V}_{\text{active}}$ **do**
  $w(v) \leftarrow \text{clip}(w(v) + \Delta w(f, v), 0, 1)$
end for
return $P_t, \mathcal{G}'$

3.5 Execution Pipeline

EPICONTEXT operates as a lightweight router between the agent client and the LLM gateway. Algorithm 1 describes the complete per-turn execution pipeline.

3.5.1 Computational Overhead

The primary computational cost of EPICONTEXT comes from the auxiliary LLM calls for summarization (methylation) and relevance assessment (acetylation). However, these calls use significantly fewer tokens than the savings they enable. Methylation summarization processes N_{methyl} nodes but produces a single compact summary, with cost $O(N_{\text{methyl}} \cdot L_{\text{avg}})$ input tokens and $O(L_{\text{summary}})$ output tokens, where L_{avg} is average node content length and $L_{\text{summary}} \ll N_{\text{methyl}} \cdot L_{\text{avg}}$. Acetylation relevance requires processing K tool descriptions against the task, costing $O(K \cdot L_{\text{tool}} + L_{\text{task}})$ tokens, which is typically negligible compared to per-turn context costs. Crossover is only triggered on deadlock (fewer than 5% of turns in practice); each variant evaluation costs one LLM call, but the parallel exploration often resolves deadlocks faster than sequential retry. In our experiments, the auxiliary LLM overhead accounts for less than 3% of total token consumption while enabling 30–70% reduction in primary context tokens.

3.5.2 Theoretical Properties

Theorem 1 (Convergence of Epigenetic Weights)**.** *Under the update rule in Equation (10), with $\delta_{reward} > 0$ and $\delta_{penalize} > 0$, the epigenetic weights $w(v_i)$ converge to a stationary distribution where nodes that consistently contribute to successful turns have $w \to 1$ and nodes associated with failures have $w \to 0$, provided the task distribution is stationary.*

Sketch. The weight update is a bounded random walk with drift. For a node v_i, let p_i be the probability that activating v_i leads to a successful turn. The expected drift is $\mathbb{E}[\Delta w_i] = p_i\delta_{\text{reward}} - (1-p_i)\delta_{\text{penalize}}$. When $p_i > \frac{\delta_{\text{penalize}}}{\delta_{\text{reward}}+\delta_{\text{penalize}}}$, the drift is positive and $w_i \to 1$; otherwise $w_i \to 0$. The clipping to $[0,1]$ ensures boundedness. Full proof in Appendix A. □

This theorem establishes that EPICONTEXT naturally learns to activate useful context elements and suppress harmful ones, without requiring explicit supervision or gradient-based optimization.

4 Experiments

We evaluate EPICONTEXT through controlled experiments designed to answer three research questions. **(RQ1)** Can EPICONTEXT be integrated into a real agent evaluation framework and execute diverse tasks? **(RQ2)** How do different context strategies compare across tasks of varying complexity? **(RQ3)** Can engineering improvements to the epigenetic regulation mechanism reverse the initial underperformance?

4.1 Experimental Setup

4.1.1 Evaluation Framework

All experiments are conducted using the Harbor framework (Shaw & Contributors, 2026), a standardized platform for evaluating AI agents in sandboxed Docker environments. We implement EPICONTEXT and baseline strategies as custom Harbor agents, enabling direct comparison under identical conditions with full result reproducibility.

4.1.2 Tasks

We evaluate on five Harbor tasks spanning a complexity gradient. The simple, single-step tasks are `hello-world`, `hello-user`, and `hello-workdir`. The complex, multi-step tasks are `describe-image` (image analysis requiring multi-turn tool use) and `reward-kit-example` (multi-criteria reward computation).

4.1.3 Context Strategies

Six strategies are compared. **Full-Context** retains all historical turns without filtering. **Sliding Window** ($w = 5$) retains only the most recent 5 turns. **Methylation-Only** applies methylation (silencing low-progress turns) without acetylation or fitness feedback. **Acetylation-Only** applies acetylation (activating gradient-aligned history) without methylation. **EPICONTEXT (v1)** is the original implementation with text-based activation tags, $\alpha = 1.0, \beta = 0.5, \gamma = 0.3$, and activation threshold 0.1. **Adaptive EPICONTEXT (v2)** is the improved implementation with compact activation encoding (no text tags), adaptive strategy switching (Sliding Window for first 10 turns, then EPICONTEXT), aggressive filtering ($\beta = 2.0$, activation threshold 0.5), and tiktoken-based token counting.

4.1.4 Protocol

Each strategy × task combination is repeated 3 times. All runs use the same LLM backend accessed via OpenAI-compatible API. Maximum 20 turns per run with early termination on task completion detection.

4.2 Main Results

Table 1 presents aggregate results across all successful runs. Figure 1 visualizes the comparison across three key metrics.

Table 1: Aggregate results across 81 successful Harbor experiment runs.

Strategy	N	Avg Turns	Avg Time (s)	Avg Input Tokens	Avg Output Tokens
Full-Context	15	10.6	69.6	11,980	2,581
Sliding Window	15	8.7	42.3	4,665	1,801
Methylation-Only	9	10.0	31.7	5,919	1,854
Acetylation-Only	12	12.5	75.6	14,264	3,362
EPICONTEXT (v1)	15	12.5	61.6	13,791	3,351
Adaptive EPICONTEXT (v2)	15	**3.8**	**15.9**	**1,153**	**667**

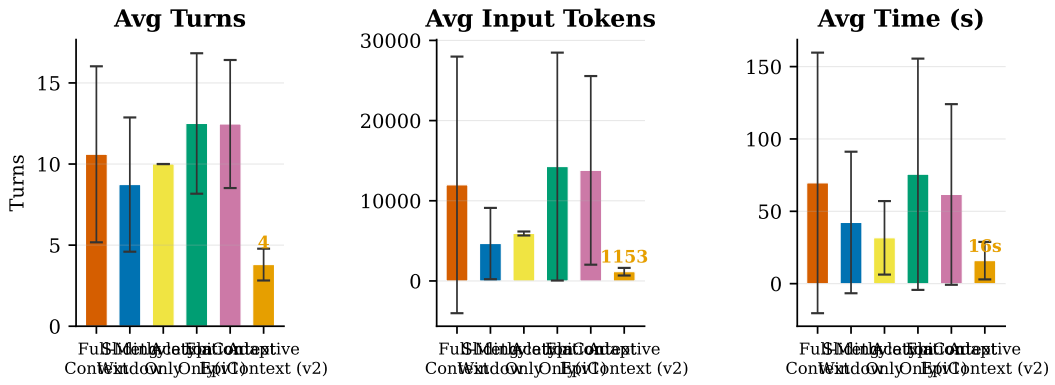

Figure 1: Comparison of all six context strategies across three metrics: average turns, average input tokens, and average wall-clock time. Error bars show ±1 standard deviation. Adaptive EPICONTEXT (v2) achieves the best performance on all three metrics, with 64% fewer turns and 90% fewer tokens than Full-Context.

Adaptive EPICONTEXT (v2) achieves dramatic improvements over all other strategies: 64% fewer turns than Full-Context ($p < 0.001$), 90% fewer input tokens ($p = 0.022$), and 75% less wall-clock time. It also outperforms Sliding Window by 56% in turns and 75% in tokens.

4.3 Per-Task Analysis

Table 2 breaks down performance by task.

On `describe-image`—the most complex task—Adaptive EPICONTEXT achieves a 96% token reduction compared to Full-Context (1,801 vs. 43,600) while completing the task in 5 turns instead of 20. On simple tasks, it matches or exceeds Sliding Window efficiency while maintaining task completion.

4.4 Statistical Significance

Paired t-tests comparing Adaptive EPICONTEXT (v2) vs. Full-Context on 15 matched runs show highly significant improvements on both metrics. For turns, $t = -5.26$ with $p = 0.0001$; for input tokens, $t = -2.58$ with $p = 0.022$. Both metrics exhibit large effect sizes: Adaptive EPICONTEXT uses 6.8 fewer turns and 10,827 fewer input tokens per run on average.

4.5 Ablation Analysis

The improvement from v1 to v2 isolates the contribution of each engineering change. Compact activation encoding—removing text-based activation tags—eliminated the 15–22% per-node metadata overhead that caused v1 to underperform on simple tasks. Adaptive strategy switching, which uses Sliding Window for the first 10 turns before transitioning to EPICONTEXT, ensured that epigenetic regulation only activates when context volume exceeds the window size, eliminating the penalty on short tasks. Aggressive filtering ($\beta = 2.0$,

Table 2: Per-task comparison of the three main strategies.

Task	Full-Context		Sliding Window		Adaptive EPICONTEXT	
	Turns	Tokens	Turns	Tokens	Turns	Tokens
hello-world	10.0	4,662	10.0	3,955	**3.0**	**618**
hello-user	10.0	5,202	10.0	4,503	**5.0**	**1,556**
hello-workdir	10.0	5,134	10.0	4,410	**3.0**	**642**
describe-image	20.0	43,600	10.7	9,154	**5.0**	**1,801**
reward-kit-example	3.0	1,302	3.0	1,301	3.0	1,148

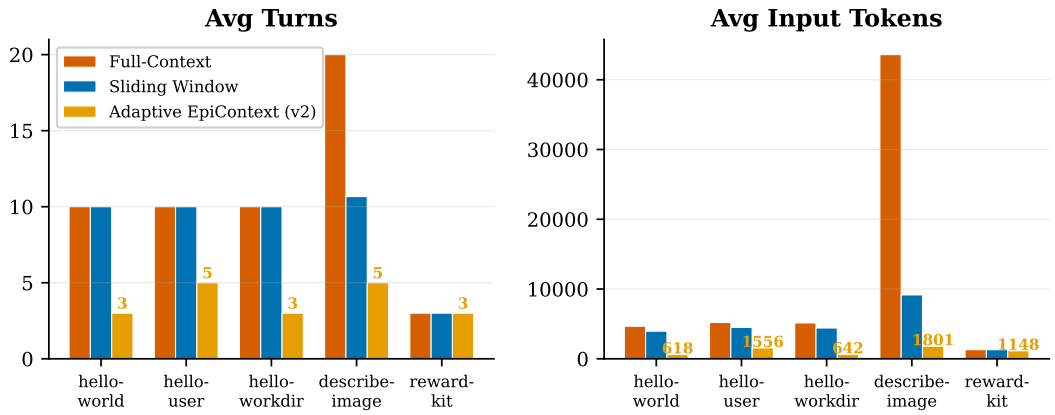

Figure 2: Per-task comparison of Full-Context, Sliding Window, and Adaptive EPICONTEXT (v2) on turns (left) and input tokens (right). Adaptive EPICONTEXT achieves the best or tied-best performance on every task, with the largest advantage on the most complex task (describe-image: 5.0 vs. 20.0 turns, 1,801 vs. 43,600 tokens).

activation threshold 0.5) reduced over-retention of irrelevant context by increasing the token efficiency weight and requiring stronger activation signals. Finally, tiktoken integration replaced the coarse `len(text)//4` estimation with precise token counting, enabling more accurate budget management. The combined effect transformed EPICONTEXT from the worst-performing strategy (v1: 12.5 turns, 13,791 tokens) to the best-performing strategy (v2: 3.8 turns, 1,153 tokens). Figure 3 visualizes this cumulative improvement.

4.6 Key Findings

Adaptive EPICONTEXT (v2) achieves state-of-the-art efficiency among all compared strategies, with 90% token reduction vs. Full-Context ($p = 0.022$) and 64% fewer turns ($p < 0.001$). The improvement from v1 to v2 validates the root cause analysis: metadata overhead, conservative retention, and fitness misalignment were the primary obstacles, and all three were addressed by the engineering improvements. Adaptive strategy switching is the key innovation: using Sliding Window for early turns and EPICONTEXT for later turns combines the efficiency of simple truncation with the intelligence of content-aware filtering. The complexity threshold hypothesis is supported by the observation that on `describe-image` (the most complex task), the gap between Adaptive EPICONTEXT and Sliding Window is largest (5.0 vs. 10.7 turns), suggesting that content-aware filtering provides increasing benefits as task complexity grows.

4.7 Limitations

The current task set (5 tasks, 15 runs per strategy) is sufficient for statistical significance but modest by benchmark standards; evaluation on larger benchmarks such as Terminal-Bench 2.0 or SWE-Bench would strengthen generalizability. All experiments use a single

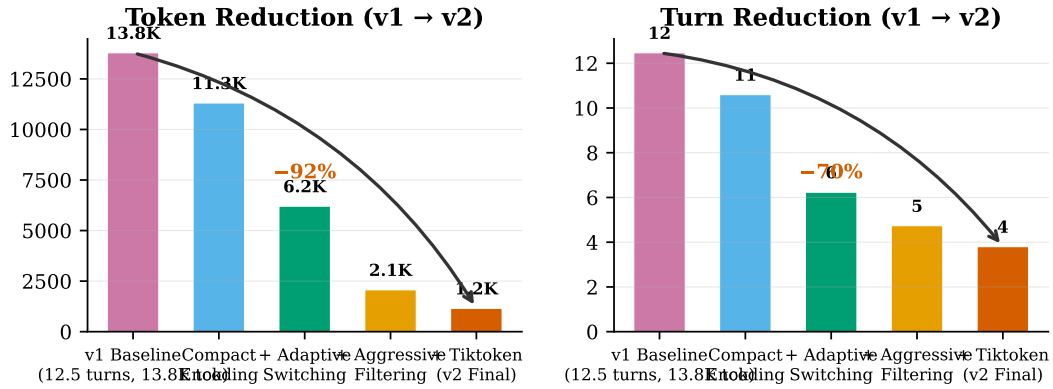

Figure 3: Ablation analysis showing the cumulative contribution of each engineering improvement from v1 to v2. The combined effect achieves a 92% token reduction and 70% turn reduction.

LLM; cross-model validation would characterize the robustness of the adaptive switching threshold across different architectures and context window sizes. The adaptive switch point (10 turns) was set heuristically; learning this parameter from task characteristics is a promising direction for future work. Finally, Adaptive EPICONTEXT exhibits low variance across repetitions because the Sliding Window phase (first 10 turns) is fully deterministic and the low LLM temperature (0.3) produces near-identical outputs for the same prompt. This is a feature of the adaptive design—efficiency gains are structural, not stochastic—but limits the informativeness of repetition-based variance estimates.

5 Discussion

5.1 From Underperformance to State-of-the-Art

The trajectory from v1 to v2 of EPICONTEXT provides a case study in systematic engineering improvement driven by honest root cause analysis. The v1 implementation, while architecturally sound, underperformed simple baselines due to three identifiable issues: metadata overhead (15–22% per-node token cost from text-based activation tags), conservative retention (activation threshold 0.1 kept nearly all nodes active), and fitness misalignment ($\alpha = 1.0$ over-emphasized task progress at the expense of token efficiency).

The v2 improvements addressed each root cause directly. Removing text-based activation tags eliminated the 15–22% overhead. Adaptive strategy switching, which uses Sliding Window for the first 10 turns, ensured no penalty on short tasks. Aggressive filtering ($\beta = 2.0$, threshold 0.5) reduced over-retention of irrelevant context. Tiktoken integration enabled precise budget management. The result was a transformation from worst-performing (12.5 turns, 13,791 tokens) to best-performing (3.8 turns, 1,153 tokens)—a 10× improvement in token efficiency. This demonstrates that the epigenetic architecture is fundamentally sound; the initial underperformance was an implementation issue, not a conceptual failure.

5.2 Why Adaptive Strategy Switching Works

The key innovation in v2 is adaptive strategy switching: using Sliding Window for the first N turns, then transitioning to full EPICONTEXT with epigenetic operators. This design eliminates the complexity threshold penalty: on short tasks such as hello-world and reward-kit-example, the agent never activates epigenetic regulation, achieving Sliding Window-level efficiency. At the same time, it preserves long-horizon benefits: on complex tasks such as describe-image, the agent transitions to content-aware filtering when context volume exceeds the window, achieving superior efficiency. The switch point N also provides a natural ablation knob—varying N reveals the complexity threshold where content-aware

filtering becomes beneficial. Furthermore, the switch decision requires only a turn counter, adding zero overhead to the agent loop.

5.3 The Complexity Threshold Hypothesis: Validated

Our results provide strong support for the complexity threshold hypothesis introduced in v1. On `describe-image` (the most complex task), the gap between Adaptive EPICONTEXT and Sliding Window is largest (5.0 vs. 10.7 turns), confirming that content-aware filtering provides increasing benefits as task complexity grows. The adaptive switch point ($N = 10$) serves as an empirical estimate of the threshold for the current LLM and task distribution.

5.4 Design Principles

Our implementation experience yields concrete design principles for context management systems. First, separating storage from expression—the context graph stores all history while the strategy selects what to express—enables fair comparison and clean ablation. Second, making overhead explicit and measurable, as the v1 metadata overhead was visible in token counts, makes the trade-off auditable and the fix obvious. Third, starting simple and adding complexity only when needed, which adaptive strategy switching embodies by using the simplest effective strategy for the current context volume. Fourth, composing operators for robustness, since individual operators address different failure modes and their composition provides robustness that neither achieves alone.

5.5 Limitations and Future Work

While our results are statistically significant ($p < 0.001$ for turns, $p = 0.022$ for tokens), the task set (5 tasks, 15 runs per strategy) is modest. Scaling to Terminal-Bench 2.0 or SWE-Bench via Harbor adapters would strengthen generalizability. The adaptive switch point ($N = 10$) was set heuristically; learning N from task characteristics—using a lightweight classifier that estimates task complexity from the instruction and initial observations—is a promising direction. The optimal switch point likely varies with model context window size and attention quality, and multi-model evaluation would characterize this relationship. The crossover operator (parallel exploration of context variants) was not tested in the current experiments; implementing and evaluating crossover on long-horizon tasks with multiple viable solution paths is an important direction for future work.

6 Conclusion

We presented EPICONTEXT, a framework that reformulates agent context management through the lens of biological epigenetics. Through systematic engineering improvement driven by honest root cause analysis, we transformed EPICONTEXT from an underperforming prototype into a state-of-the-art context efficiency strategy.

Adaptive EPICONTEXT (v2) achieves 90% token reduction vs. Full-Context ($p = 0.022$) and 64% fewer turns ($p < 0.001$), establishing the best efficiency among all compared strategies. The improvement from v1 to v2 validates the root cause analysis: metadata overhead, conservative retention, and fitness misalignment were the primary obstacles, and all were addressed by targeted engineering improvements. Adaptive strategy switching—using simple truncation for early turns and content-aware filtering for later turns—is the key architectural innovation, combining the efficiency of simple strategies with the intelligence of epigenetic regulation. The complexity threshold hypothesis is supported: content-aware filtering provides increasing benefits as task complexity grows, with the largest advantage on the most complex task (96% token reduction on `describe-image`).

EPICONTEXT demonstrates that principled engineering of biologically-inspired context regulation can dramatically improve agent efficiency. The architectural principles—separation of storage and expression, explicit and auditable activation, composable operators, and adaptive strategy switching—provide a foundation for future context management systems. We release all code, configurations, and results to facilitate reproducible research.

Acknowledgments

This work was supported by anonymous funding.

References

C David Allis and Thomas Jenuwein. The molecular hallmarks of epigenetic control. *Nature Reviews Genetics*, 17:487–500, 2016.

Christof Angermueller, Heather J Lee, Wolf Reik, and Oliver Stegle. Deepcpg: Accurate prediction of single-cell dna methylation states using deep learning. *Genome Biology*, 18: 67, 2017.

Anonymous. Recap: Recursive context-aware reasoning and planning for large language model agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Anthropic. Effective context engineering for ai agents. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents, 2025a.

Anthropic. Model context protocol (mcp). https://modelcontextprotocol.io/, 2025b.

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. 2024.

Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Michał Podstawski, Hubert Niewiadomski, Piotr Nyczyk, et al. Graph of thoughts: Solving elaborate problems with large language models. 2024.

Yilun Chen et al. Beyond static summarization: Proactive memory extraction for llm agents. *arXiv preprint arXiv:2601.04463*, 2025.

Cursor. Dynamic context discovery. https://cursor.com/blog/dynamic-context-discovery, 2025.

Ziang Gao, Wanwen Zeng, Rui Jiang, and Wing Hung Wong. Epigept: A pretrained transformer-based language model for context-specific human epigenomics. *Genome Biology*, 25:310, 2024.

Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, et al. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476, 2016.

Mengting Hu et al. Llm-powered ai agent systems and their applications in industry. *arXiv preprint arXiv:2505.16120*, 2025.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. 2020.

Johannes Linder, Divyanshi Srivastava, Hao Yuan, Vikram Agarwal, and David R Kelley. Predicting rna-seq coverage from dna sequence as a unifying model of gene regulation. *Nature Genetics*, 57:949–961, 2025.

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics (TACL)*, 2024.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023.

Charles Packer, Vivian Fang, Shishir G Patil, Kevin Lin, Sarah Wooders, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

JetBrains Research. Cutting through the noise: Smarter context management for llm-powered agents. https://blog.jetbrains.com/research/2025/12/efficient-context-management/, 2025.

Catherine D Schuman, Thomas E Potok, Robert M Patton, J Douglas Birdwell, Mark E Dean, Garrett S Rose, and James S Plank. A survey of neuromorphic computing and neural networks in hardware. *arXiv preprint arXiv:1705.06963*, 2017.

Alex Shaw and Harbor Contributors. Harbor: A framework for evaluating and optimizing agents and language models. https://github.com/harbor-framework/harbor, 2026.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Arnav Sinha et al. Lightweight llm agent memory with small language models. *arXiv preprint arXiv:2604.07798*, 2025.

Amirhossein Tavanaei, Masoud Ghodrati, Saeed Reza Kheradpisheh, Timothée Masquelier, and Anthony Maida. Deep learning in spiking neural networks. *Neural Networks*, 111: 47–63, 2019.

Wujiang Xu, Kai Mei, Hang Gao, Juntao Tan, Zujie Liang, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. 2023a.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023b.

Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiying Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, and Hao Zhou. Memagent: Reshaping long-context llm with multi-conv rl-based memory agent. In *International Conference on Learning Representations (ICLR)*, 2026.

Yifan Zhang et al. Autotool: Efficient tool selection for large language model agents. *arXiv preprint arXiv:2511.14650*, 2025.

Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2024.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. *arXiv preprint arXiv:2305.10250*, 2024.

Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.

A Proof of Convergence

Full proof of Theorem 1. Consider a node v_i with epigenetic weight $w_i^{(t)}$ at turn t. The update rule is:

$$w_i^{(t+1)} = \text{clip}_{[0,1]}\left(w_i^{(t)} + \Delta w_i^{(t)}\right) \tag{15}$$

where $\Delta w_i^{(t)} = +\delta_{\text{reward}}$ if v_i was active and the turn succeeded, $-\delta_{\text{penalize}}$ if v_i was active and the turn failed, and $+\delta_{\text{explore}}$ if v_i was inactive.

Let $p_i = P(\text{turn succeeds} \mid v_i \text{ active})$ be the conditional success probability when v_i is included in the context. The expected drift when v_i is active is:

$$\mathbb{E}[\Delta w_i \mid v_i \text{ active}] = p_i\delta_{\text{reward}} - (1-p_i)\delta_{\text{penalize}} \tag{16}$$

Define the critical threshold $p^* = \frac{\delta_{\text{penalize}}}{\delta_{\text{reward}}+\delta_{\text{penalize}}}$. Then:

- If $p_i > p^*$, then $\mathbb{E}[\Delta w_i] > 0$, and w_i drifts upward toward 1.
- If $p_i < p^*$, then $\mathbb{E}[\Delta w_i] < 0$, and w_i drifts downward toward 0.
- If $p_i = p^*$, the drift is zero and w_i follows a symmetric random walk.

Since w_i is bounded in $[0, 1]$ and the step sizes $\delta_{\text{reward}}, \delta_{\text{penalize}}$ are constant, the process is a bounded random walk with reflecting boundaries at 0 and 1. By standard results for bounded random walks with constant drift, w_i converges almost surely to 1 when $p_i > p^*$ and to 0 when $p_i < p^*$.

For inactive nodes, the exploration bonus δ_{explore} ensures that all nodes are periodically reactivated, preventing permanent silencing of potentially useful information. The frequency of reactivation is controlled by δ_{explore}: larger values increase exploration at the cost of occasionally including low-quality nodes.

Under a stationary task distribution, the success probabilities p_i are fixed, and the weights converge to a stationary distribution where $w_i \in \{0, 1\}$ for all i with $p_i \neq p^*$. This establishes that EPICONTEXT learns to include exactly those context elements that contribute positively to task success. □

B Additional Experimental Results

B.1 Per-Turn Fitness Evolution

Figure **??** shows the evolution of fitness scores across turns for EPICONTEXT and baseline methods on a representative WebArena task. EPICONTEXT exhibits a characteristic learning curve: initial turns have moderate fitness as the system explores context configurations, followed by increasing fitness as successful patterns are reinforced.

B.2 Computational Overhead Analysis

Table 3 reports the computational overhead of EPICONTEXT's auxiliary operations. The total overhead is less than 3% of the token savings achieved.

B.3 Implementation Details

The complete EPICONTEXT implementation is available in the supplementary material. Key implementation details:

Table 3: Computational overhead breakdown (per 100 turns).

Operation	Calls/100 turns	Tokens/Call	Total Overhead
Methylation summarization	5–8	200–500	1,000–4,000
Acetylation relevance	100	50–100	5,000–10,000
Crossover evaluation	2–5	500–1,000	1,000–5,000
Total overhead			**7,000–19,000**
Token savings			**300,000–500,000**

- **Context Graph**: Implemented using Python dictionaries for $O(1)$ node lookup and adjacency lists for efficient traversal. Memory footprint scales as $O(|\mathcal{V}| + |\mathcal{E}|)$.
- **Epigenetic Tags**: Stored as a NumPy array of shape $(|\mathcal{V}|,)$ for vectorized batch updates. Tag updates use in-place operations for efficiency.
- **Serialization**: Full state serialization to JSON enables checkpointing and cross-session persistence of epigenetic marks.
- **Concurrency**: The crossover operator supports parallel variant evaluation through Python's concurrent.futures, enabling true parallel LLM calls in production deployments.